

University Examination Management System

Final software engineering documentation | Revised edition

A university application for controlled exam participation, grading and grade review.

Prepared by Koorosh Ghavidel Taghavi

University ID 40114141054080

Supervisor Dr.Javad Shahparian

Stack Next.js, React, TypeScript, Express, Prisma and PostgreSQL

Purpose Explain the approved requirements, domain design, workflows and validation plan in a form suitable for academic review.

Evidence boundary This revision uses the supplied documentation and approved project decisions. The current source repository, migrated schema and running application have not been inspected. Diagrams are precise design views; they are not claims of verified implementation. API families, symbolic statuses and suggested constraints must be reconciled with the code before being presented as an implementation reference.

Revision focus Clear cardinalities, service responsibilities, deadline branches, publication controls and atomic grade changes. All diagrams use vector graphics for readable zoom and printing.

Contents and reading guide

Part	Coverage
1-3	Problem, objectives, actors and scope
4-6	Functional and nonfunctional requirements; business rules
7	Use cases and exceptional paths
8	Architecture, component and deployment diagrams
9	Database relationships and domain class diagrams
10-11	API access policy, authentication and security
12	Sequence, activity and lifecycle diagrams
13-16	Testing, deployment, future work and conclusion

The following sections retain the original document structure while expanding the technical figures. Figure captions explain the relevant invariant and the limit of the model.

Diagram conventions Entity boxes show selected fields rather than every column. ER connectors carry minimum and maximum cardinality at both ends. Domain class connectors describe associations, not database foreign keys. Activity decision boxes are labelled as decisions. State labels describe business states and may differ from enum names in the implementation.

Unresolved policy An accepted descriptive upload that has not been explicitly submitted by cutoff needs a stated policy. This document shows that case separately and does not silently classify the student as absent. The exact interpretation of six calendar months and rules for repeated objections also require confirmation in code and university policy.

1 Introduction and problem statement

The system supports university examinations from academic setup to student participation, grading, publication and objections. It replaces disconnected handling of exam schedules, answer files and grade review with a course scoped workflow.

The central problem is preserving academic integrity across eligibility, time limits and ownership. A valid login alone does not authorize access to an exam or submission. Every operation must be checked against the target course, exam, attempt or grade.

The intended result is a Persian web application in which eligible students take exams and assigned professors manage assessment. The server controls deadlines and scores, and records important grading decisions.

This report documents the approved design. Successful migration and local runtime startup were reported in project work, but do not establish that all workflows or security tests passed.

2 Objectives and scope

- Provide university provisioned accounts without public self registration.
- Manage course offerings, enrollments and professor assignments through administrative setup.
- Support multiple choice and descriptive exams, strict deadlines and private file access.
- Separate grade calculation or entry from explicit publication.
- Support grade objections within the approved six month window and preserve meaningful grade revisions.
- Provide published result views, course comparisons and performance trends.

Scope boundary Online proctoring, single sign on, notifications, special accommodations and automatic descriptive marking are future proposals. Account setup does not grant administrators unrestricted grade editing or objection review.

3 Stakeholders and actors

Actor	Responsibility
Student	View eligible exams; save answers or upload files; submit; view own published grades; submit and track objections.
Professor	Manage assigned course exams; review submissions; enter descriptive scores; publish results; respond to objections.
Administrator	Provision accounts, courses, enrollments and professor course links.
Academic office	Define assessment policy, schedules and review responsibilities.
Operator	Maintain application, database, private file storage and backups.

The deadline processor is an internal system responsibility, not a human role. Its concrete scheduling mechanism must be verified in deployment configuration.

4 Functional requirements

ID	Required behavior
FR01	Authenticate active provisioned users; refresh and revoke sessions.
FR02	Maintain academic setup and role scoped course lists.
FR03	Create, edit and remove eligible draft exams; validate publication; enforce professor assignment.
FR04	Expose published exams only to eligible enrolled students.
FR05	Start or resume one attempt; provide ordered MCQ questions and options without the answer key.
FR06	Save selected options; support previous, next and numbered question navigation; calculate score on the server.
FR07	Download descriptive questions and accept PDF, JPG, JPEG or PNG answer files through authorized handlers.
FR08	Reject late writes; finalize MCQ using saved answers; mark descriptive nonattendance when no accepted answer exists.
FR09	Review submissions; enter scores within exam bounds; publish grades explicitly.
FR10	Display own published results, course comparisons and trends.
FR11	Create and track eligible objections; professor responds and keeps or revises grade.
FR12	Preserve revision actor, reason, prior score, new score and time.

5 Nonfunctional requirements and assumptions

Area	Design requirement
Security	Authenticate every protected request and authorize its target; isolate student and professor data.
Integrity	Use transactions, unique constraints and atomic status changes for finalization and grade revision.
Time	Store absolute instants; convert Solar Hijri UI values with Asia/Tehran; use trusted server time.
Usability	Persian RTL interface, readable countdown, clear upload status, accessible errors and final submission confirmation.
Reliability	Durable records, recoverable backups and a deadline process that can retry safely.
Performance	Index eligibility and ownership lookups; paginate lists; impose configured upload bounds.
Assumptions	University supplies accounts and academic links. The design permits one attempt per student per exam. Numeric service targets and upload limits are not approved here.

6 Business rules

- An active course enrollment is required for student exam access. Professor management requires the applicable course assignment and exam responsibility.
- Publishing validates schedule, exam content and point allocation. Changes to published content must not invalidate existing attempts.
- For an attempt started at t , $\text{cutoff} = \min(\text{exam end}, t + \text{duration})$ when duration applies; otherwise $\text{cutoff} = \text{exam end}$. Accept writes only when server time is strictly before cutoff.
- At MCQ cutoff, score committed answers; missing answers remain unanswered. A late client response cannot extend the cutoff.
- A descriptive attempt with no accepted answer file at cutoff is absent. A file is accepted only after validation and durable association before cutoff.
- Student payloads omit correct option flags and unpublished grades. Finalized attempts are immutable.
- Grades become visible through explicit professor publication. Objection eligibility starts from grade publication and lasts six months under the confirmed calendar policy.
- Grade revision and objection resolution are recorded atomically with actor, reason and response. File retention policy is separate from the objection window.

Policy to resolve Define the treatment of accepted but unsubmitted descriptive files, repeated objections, and calendar month calculation before claiming these paths are complete.

7 Use cases

ID	Actor and use case	Success outcome
UC01	All roles · Sign in	Role scoped authenticated session
UC02	Admin · Configure academic setup	Accounts and academic relationships stored
UC03	Professor · Prepare and publish exam	Valid exam visible to eligible students
UC04	Student · Take MCQ exam	Finalized answers and backend score
UC05	Student · Submit descriptive exam	Validated private files linked to final attempt
UC06	Professor · Review and publish grade	Published result visible to its student
UC07	Student · Object to grade	Eligible review request recorded
UC08	Professor · Resolve objection	Response and optional grade revision preserved

UC01 An active account supplies credentials. The server checks its password hash, creates a session and returns permitted identity information. Invalid credentials receive a generic failure. Refresh rotates credentials; logout revokes the affected session.

UC02 An authenticated administrator establishes course offerings and role relationships. Duplicate identifiers and invalid foreign references fail validation. Academic history must not be deleted as a side effect of setup changes.

7 Detailed examination use cases

UC03 Prepare and publish

Precondition: authenticated professor with the applicable assignment. Configure type, schedule, duration and points. MCQ questions require valid options and a valid answer key under the single correct option design. Descriptive exams require valid question content or file. Validate before publication. Reject unassigned access and invalid scheduling; preserve draft state on failure.

UC04 Take MCQ

Precondition: published exam, active enrollment and open start window. Create or retrieve the unique attempt and fixed question order. Return public question fields. Validate every selected option against the attempt question, ownership, status and cutoff. Finalization locks or atomically changes status, reads committed answers and computes score once. A duplicate finalization returns the existing outcome; a late save fails.

UC05 Submit descriptive

Precondition: the same eligibility and time checks. Authorize question download. Validate answer extension, MIME, content signature and configured size. Persist privately and recheck cutoff before recording acceptance. Final submission requires an accepted linked file. Upload failures do not create a valid answer. With no accepted file at cutoff, record absence; an accepted but unsubmitted file follows the separately confirmed policy.

7 Detailed grading and objection use cases

UC06 Review and publish

Precondition: professor owns or is responsible for the exam under the assignment model.

Retrieve authorized submissions and private files. MCQ scoring is automatic; descriptive scoring is entered by the professor. Validate $0 \leq \text{score} \leq \text{maximum}$. Draft grades remain hidden.

Publication records its timestamp and exposes only the student's own result.

UC07 Submit objection

Precondition: the student owns a published grade and the publication based six month window remains open. Record the student's message and initial status. Reject wrong ownership, unpublished grades and expired requests. Duplicate review rounds follow an explicit policy rather than an assumed rule.

UC08 Resolve objection

Precondition: the assigned professor can review the target exam. Open the objection, review the original submission, provide a response and keep or change the grade. A changed score must stay within bounds. One transaction writes the decision, optional grade revision and resulting objection state. The student can then view the response and updated published result.

8 Architecture and technology stack

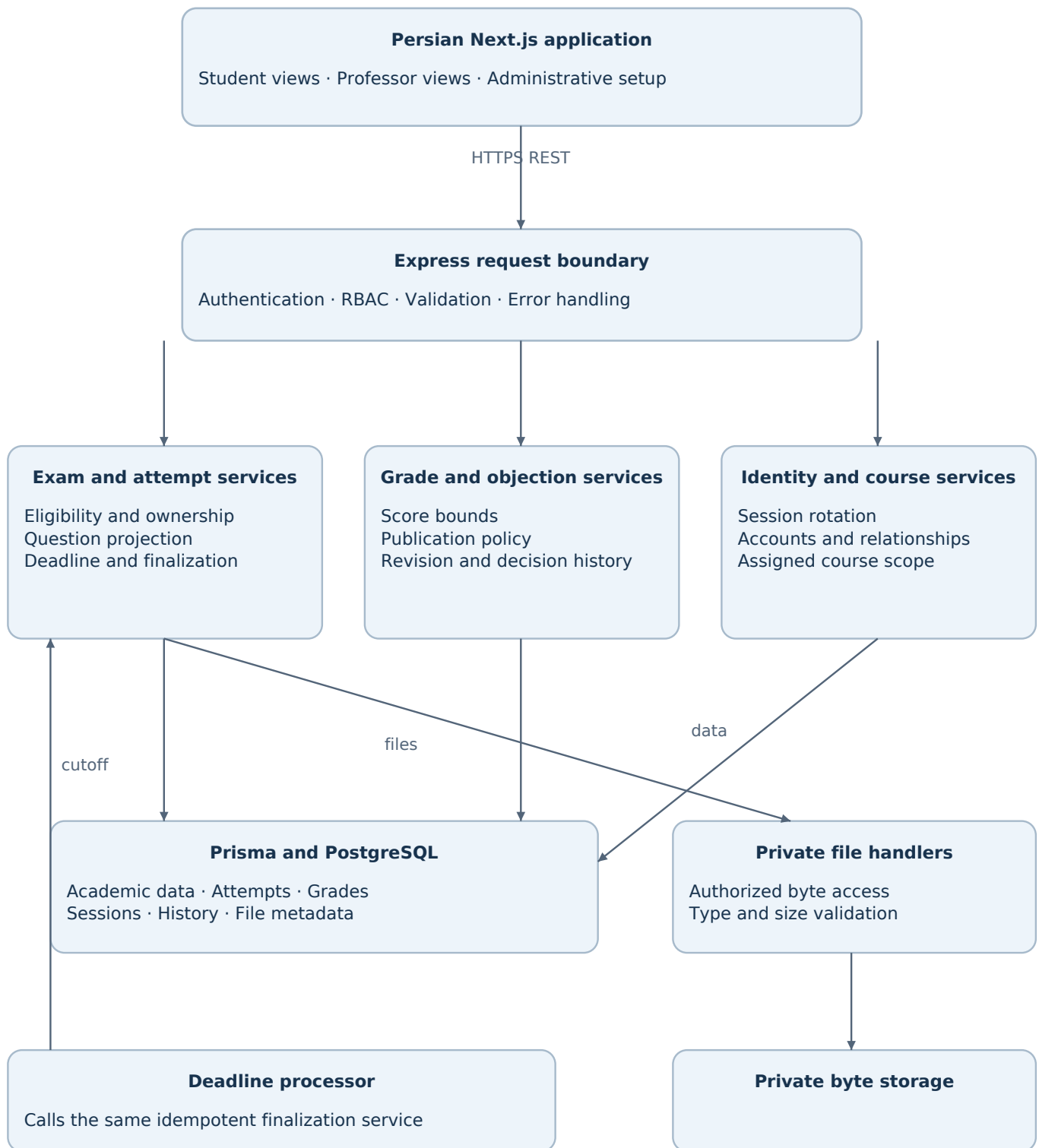
The Persian Next.js client calls an Express REST API written in TypeScript. Controllers adapt HTTP input and output. Middleware handles authentication, role checks, validation and error mapping. Services enforce course ownership, deadlines and business transitions. Prisma mediates relational persistence in PostgreSQL.

Files are private resources. The database stores metadata and academic relationships; private storage holds bytes. Download handlers resolve file identifiers through authorized exam or attempt relationships rather than trusting user supplied paths.

Layer	Technology and responsibility
Presentation	Next.js, React, TypeScript, Tailwind CSS; RTL and Solar Hijri display
API	Node.js, Express, TypeScript; controllers and middleware
Domain	Auth, courses, exams, attempts, grading and objections services
Persistence	Prisma and PostgreSQL; relational integrity and transactions
Identity	JWT access and refresh strategy; secure password hashing
Operations	HTTPS, private files, deadline processing, backups and secret configuration

Figure 1 Component architecture

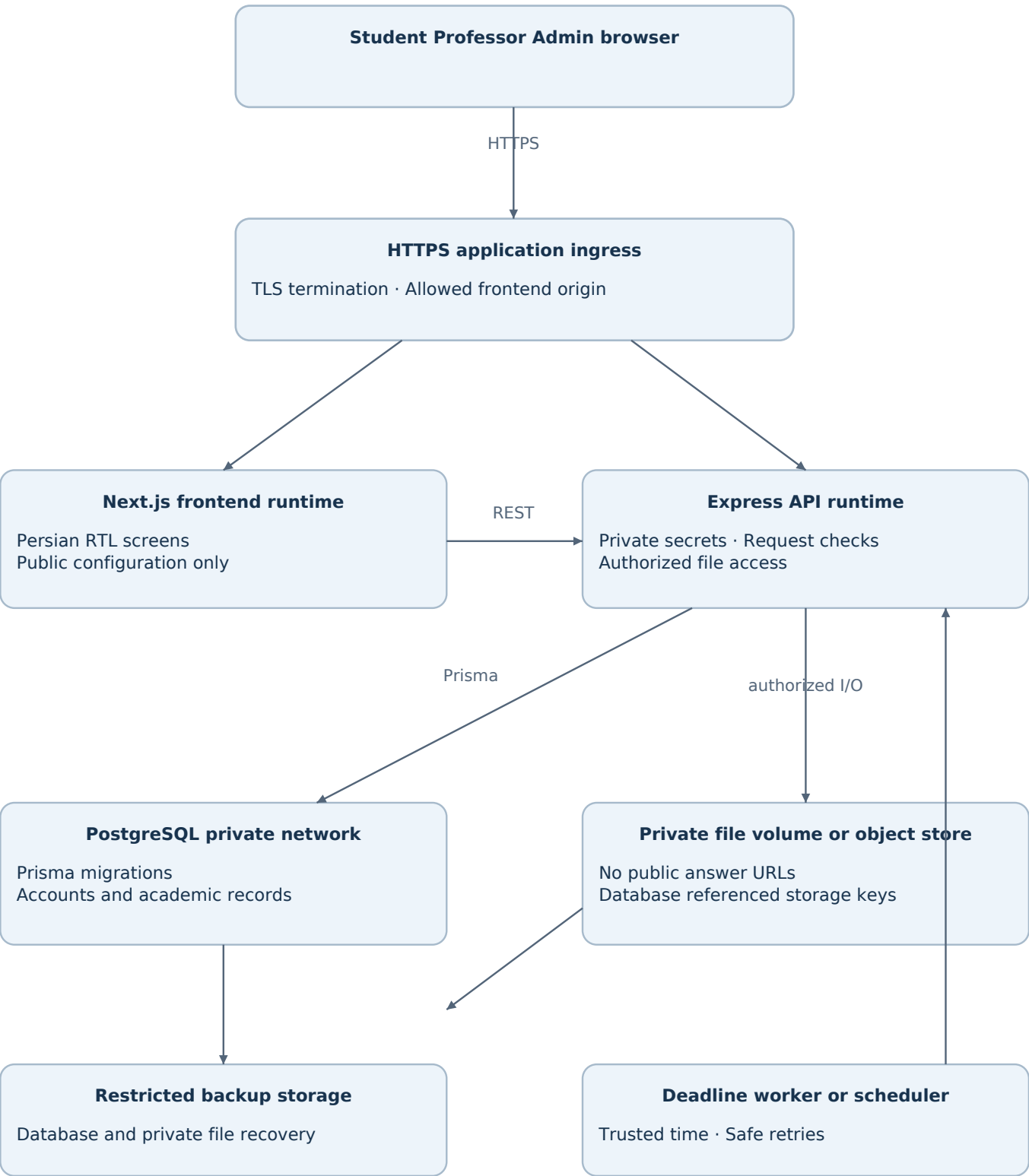
Design topology | Dependencies show who performs validation and persistence.



The processor implementation is not verified. Request time checks remain mandatory even when scheduled processing is delayed. Clients never communicate directly with private storage or PostgreSQL.

Figure 2 Deployment topology

Required environment | This is not evidence of an existing deployment.



Frontend and API may share one host in a university demonstration. The logical private boundaries still apply. Configure runtime versions, upload bounds, job scheduling and backup restore instructions using the actual environment.

9 Database design

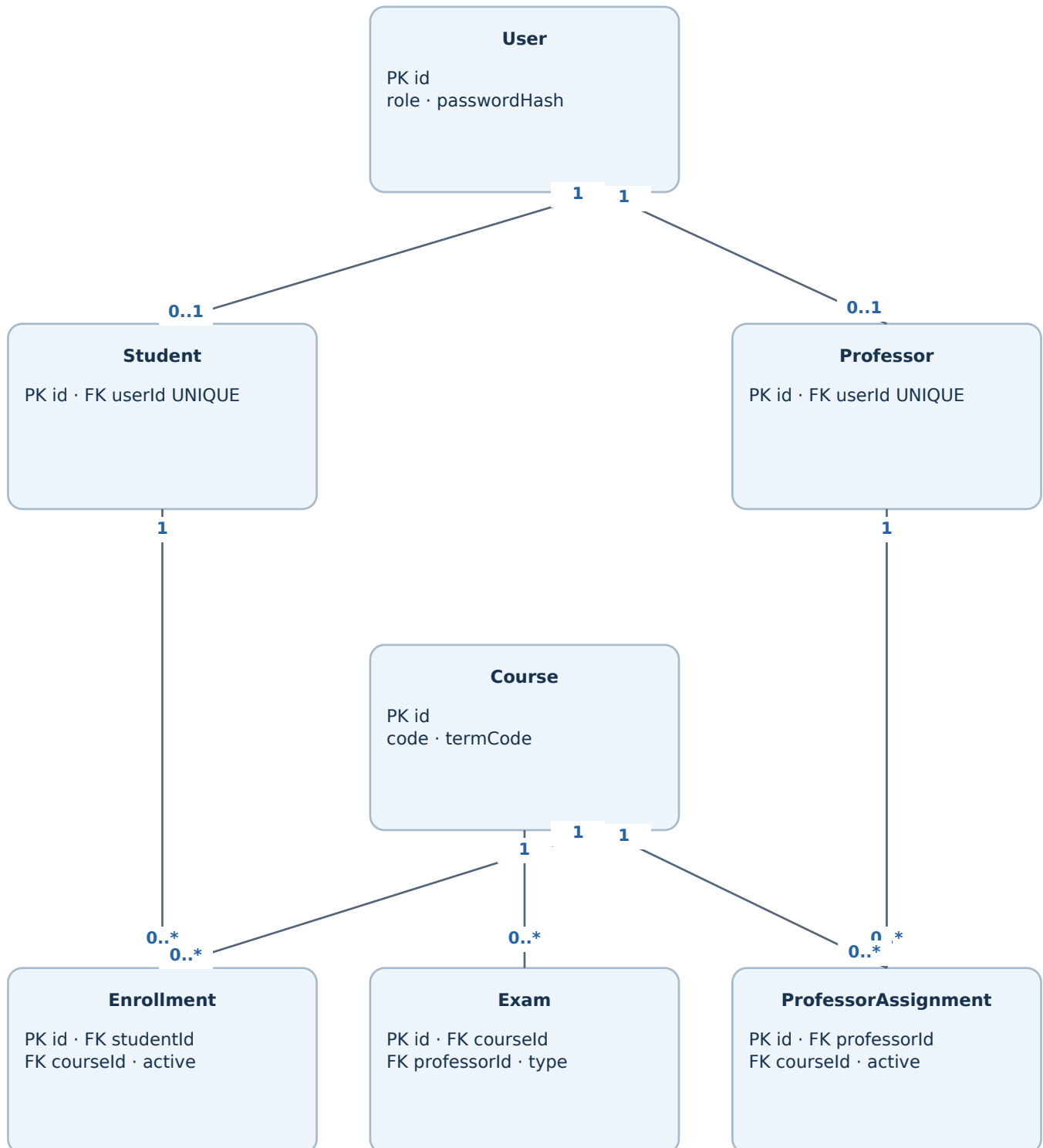
The original design identifies User, Student, Professor, Course, Enrollment, ProfessorAssignment, Exam, ExamQuestion, MultipleChoiceOption, ExamAttempt, AttemptQuestion, StudentAnswer, StoredFile, Grade, GradeRevision, GradeObjection, ObjectionDecision and AuditEvent. The following diagrams show those design concepts; exact field names and indexes require the current Prisma schema.

Invariant	Required enforcement
One attempt per exam and student	UNIQUE(examId, studentId)
Stable placement	UNIQUE(attemptId, questionId); order unique within attempt
At most one answer per placed question	UNIQUE(attemptQuestionId); option must belong to that question
One grade per attempt	UNIQUE(attemptId)
Academic scope	Exam professor and enrollment must match the exam course; validate using the relevant assignment
Historical integrity	Restrict deletion of referenced academic records; preserve grade revisions
Score integrity	Validate question points and grade range; finalization and revisions use transactions

Foreign keys alone do not guarantee cross relationship consistency. An option can exist yet belong to another question. Enforce these invariants with suitable composite constraints where practical and explicit service validation.

Figure 3 Academic ER model

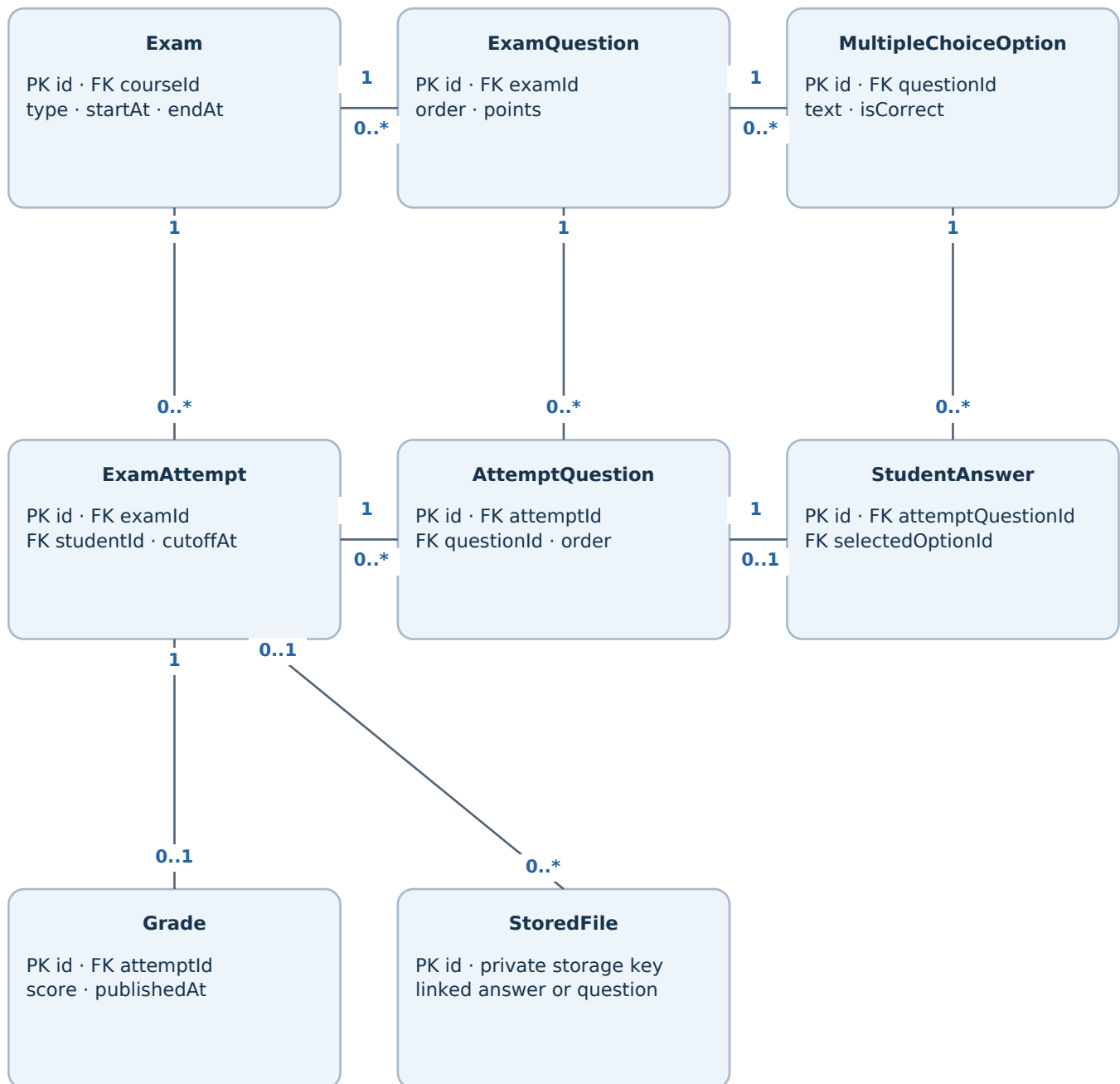
Cardinality labels: 1 = exactly one; 0..1 = optional one; 0..* = zero or many.



A student or professor profile belongs to one user. Role and profile consistency must be enforced. An exam references its responsible professor; that professor must hold the applicable course assignment. Enrollment and assignment are relationship entities, not direct many-to-many links.

Figure 4 Exam and attempt ER model

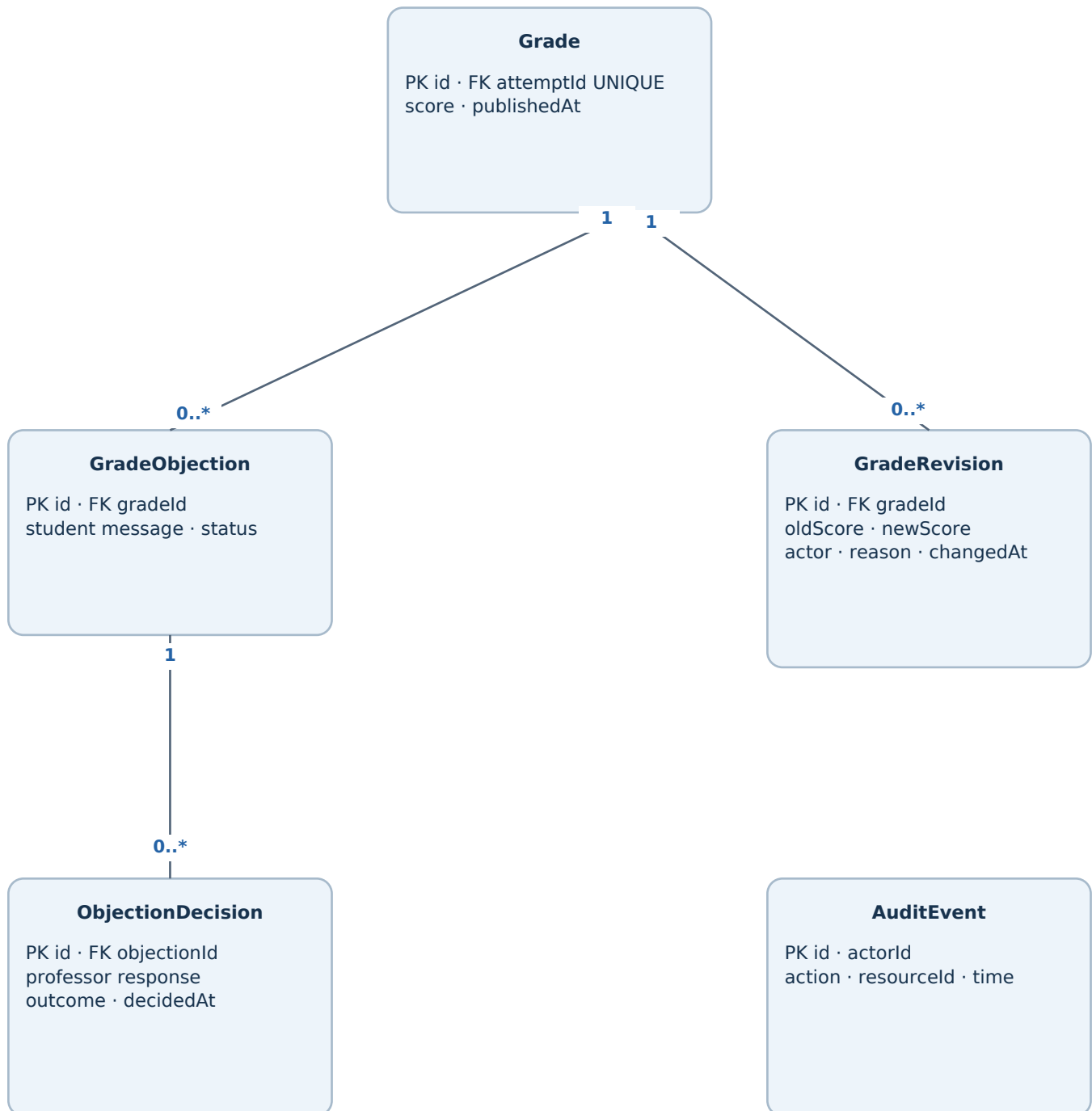
Selected fields | Question identity and option membership must remain consistent across the attempt.



An absent answer row represents an unanswered MCQ question. A saved answer references one valid option for its placed question. StoredFile ownership is conceptual here: exam question files and attempt answer files have different links. Exact nullable foreign keys and file count policy must be checked in the schema.

Figure 5 Grading and review ER model

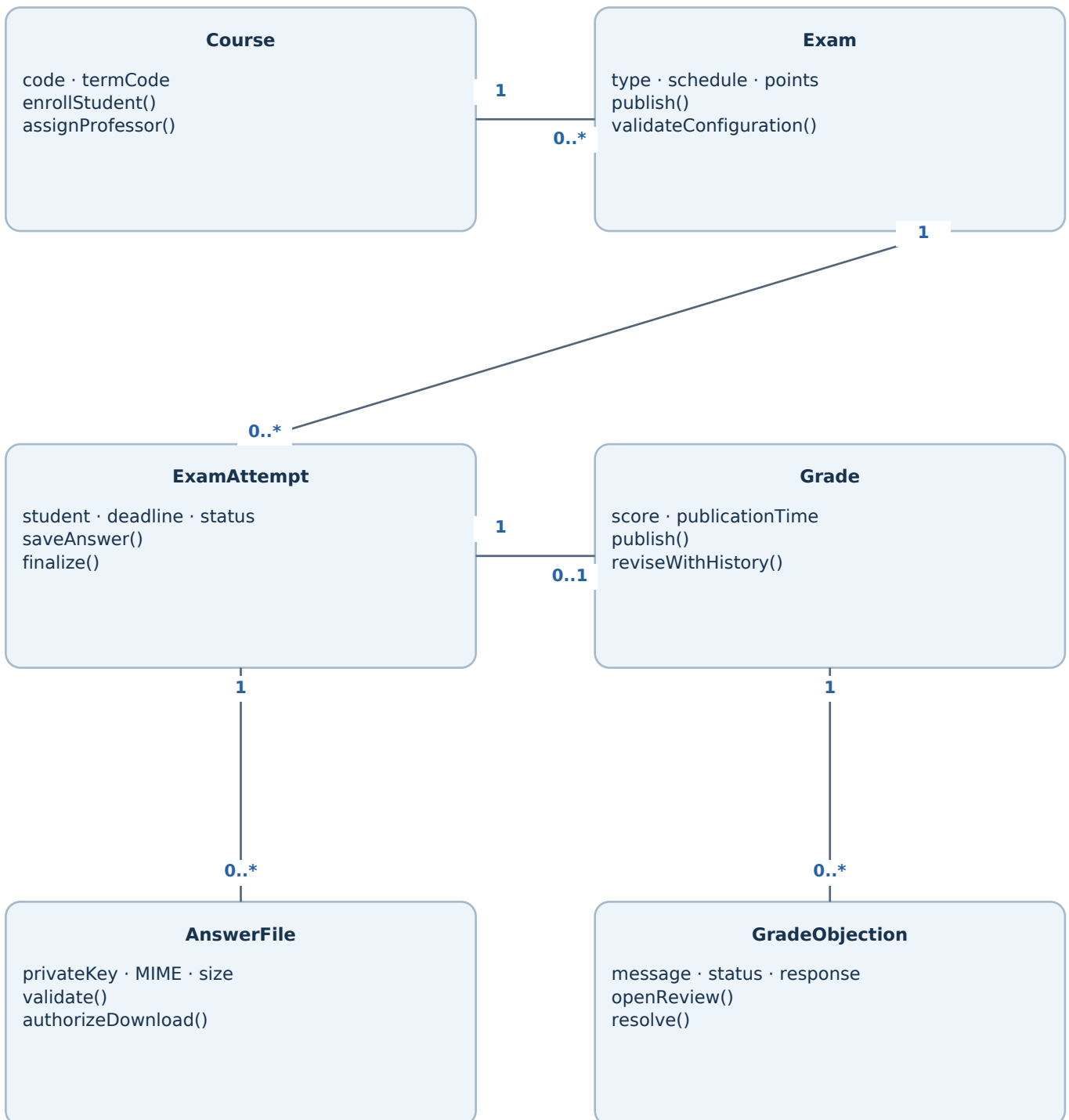
Objections and revisions are distinct records: a response can keep the score unchanged.



The design allows history; the permitted number of objections and decisions per review round is a separate business constraint. Decision, changed grade and revision must commit together. Audit events add operational traceability and do not replace GradeRevision. No unsupported foreign key between decisions and revisions is asserted.

Figure 6 Domain class model

Conceptual UML class view | Operations belong to domain behavior and need not be methods on Prisma records.



MCQ answers and descriptive files are different submission representations. Services invoke these responsibilities with authenticated actor context. Relationship multiplicities describe the domain; this view deliberately omits framework inheritance and unverified source filenames.

10 API architecture and access policy

API resource families are auth, courses, exams, questions, attempts or submissions, files, grades and objections. Exact routes, payloads and status codes require repository inspection. This report does not invent a production API contract.

Resource	Student	Professor	Admin
Courses and exams	Own active enrollments; eligible published exams	Assigned courses; responsible exams	Academic setup
Attempts and files	Own attempt; authorized question download	Submissions and files of responsible exams	No general submission authority implied
Grades	Own published grades	Review and publish on responsible exams	No general grade editing implied
Objections	Own eligible published grade	Review objections on responsible exams	No review authority implied

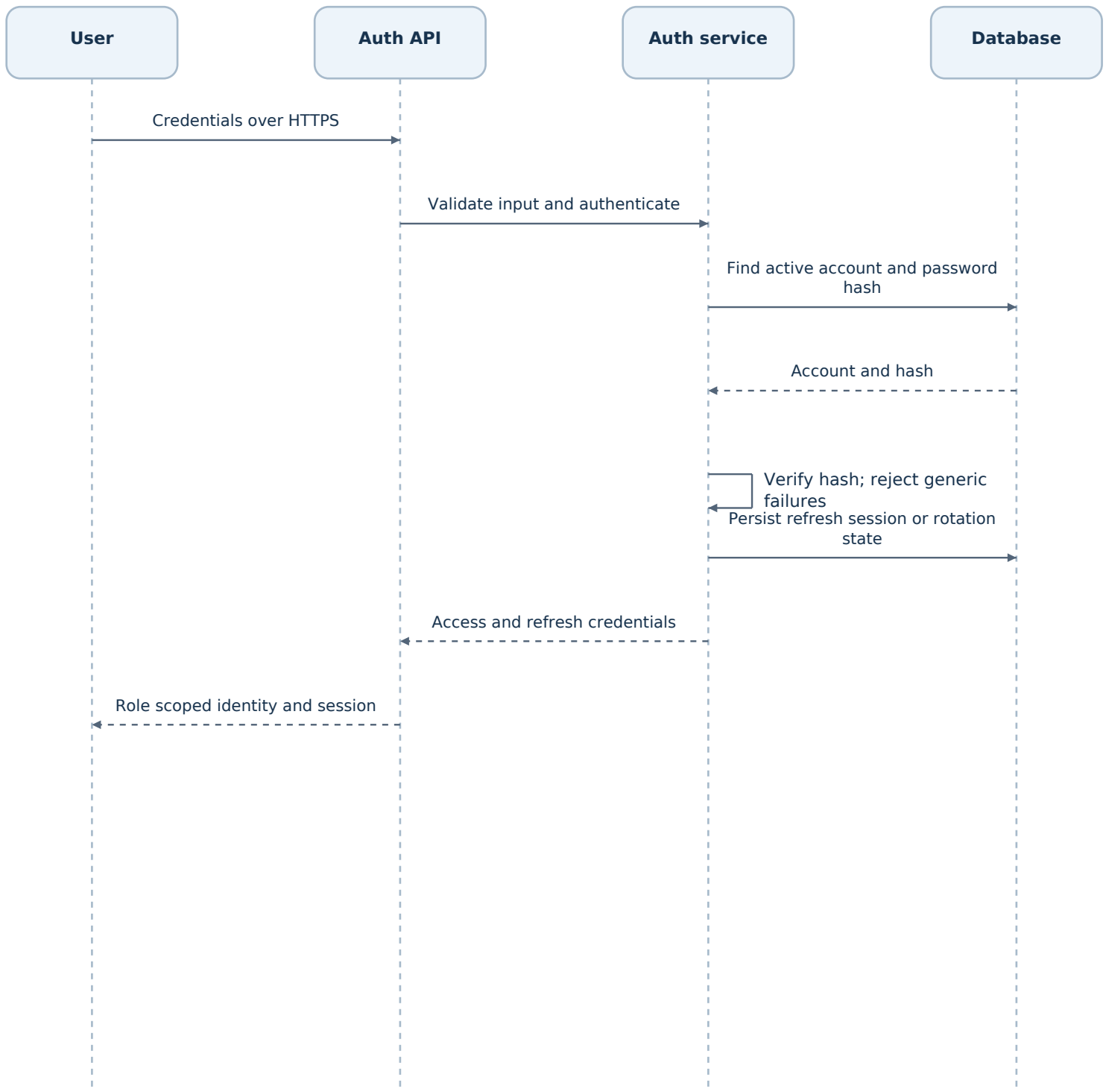
Controllers parse validated inputs and call services. Services authorize the target record through its relationships. Use explicit student response projections. Represent validation, authentication, forbidden access, not found, state conflict and closed window errors consistently without leaking secret data.

11 Authentication authorization and security

- Verify password hashes and active account state; issue limited lifetime access credentials and rotate refresh credentials. Store revocable session information and reject refresh reuse according to the implemented strategy.
- Check role and resource ownership on every protected operation. IDs in a request are references to validate, not proof of authorization.
- Use parameterized Prisma queries. Avoid SQL assembled from untrusted input. Encode rendered user content and restrict allowed browser origins.
- If browser authentication uses cookies, configure secure attributes and CSRF protection. The token transport must be confirmed before prescribing a concrete CSRF implementation.
- Validate extension, MIME, content signature and size for uploads. Generate storage keys, keep bytes private and recheck deadline after upload processing.
- Serialize answer save and finalization against the attempt. A finalized status rejects edits. Compute MCQ scores from server stored answers and the private key.
- Rate limit login and sensitive routes; keep secrets out of repository and logs. Record actionable failures without credential disclosure.
- Security validation must include student to student and professor to professor IDOR, unpublished grade exposure, answer key leakage and score manipulation.

Figure 7 Login and session sequence

UML sequence view | Solid arrows are requests; dashed arrows are responses. Ordered from top to bottom.



Refresh must validate the existing credential, rotate it atomically and detect reuse under the actual session design. Logout revokes the relevant session; existing access JWT behavior follows its expiry or revocation policy.

12 Student and professor workflows

Students sign in and view eligible upcoming, active and completed exams. They start or resume an attempt, save MCQ responses or upload descriptive answers, confirm submission and view its status. Grades appear only after publication. A published grade may be challenged within the approved window.

Professors sign in to assigned courses, prepare valid exam content, publish exams and review their submissions. MCQ results are calculated on the backend. Descriptive work is scored by the professor. The professor publishes grades and responds to authorized objections.

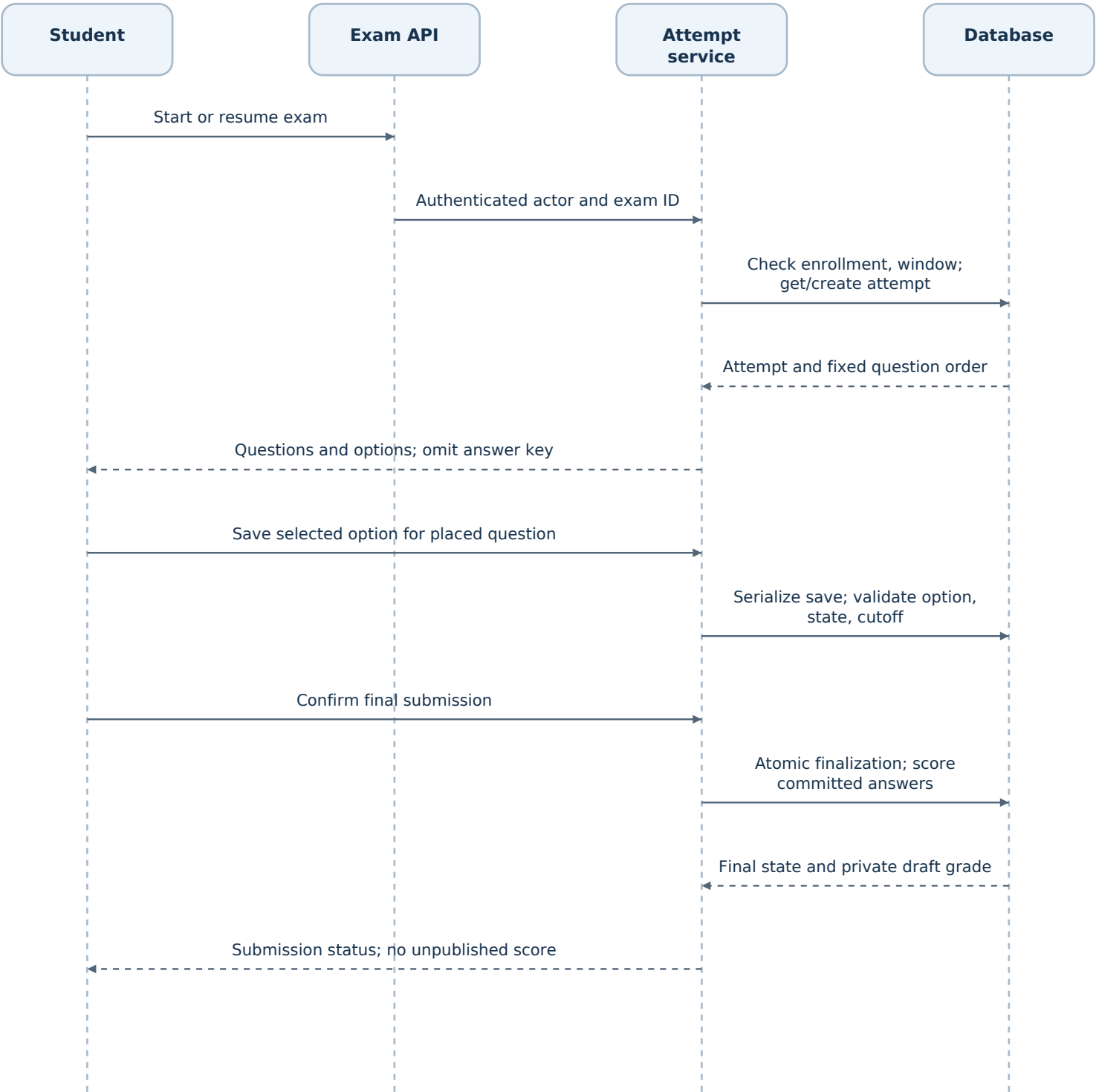
Administrators provision the accounts and relationships that make these workflows possible. Exam management and grading remain governed by academic assignment and resource scope.

Deadline semantics The server distinguishes client submission from cutoff processing. A late client request cannot be accepted merely because the worker has not run. All paths use the same idempotent finalization logic and committed answer data.

Diagram interpretation The following sequence diagrams show successful interactions and state the failure branches in captions. Activity diagrams expand the deadline and eligibility branches. Lifecycle figures separate persistent state from the time derived availability of a published exam.

Figure 8 MCQ participation sequence

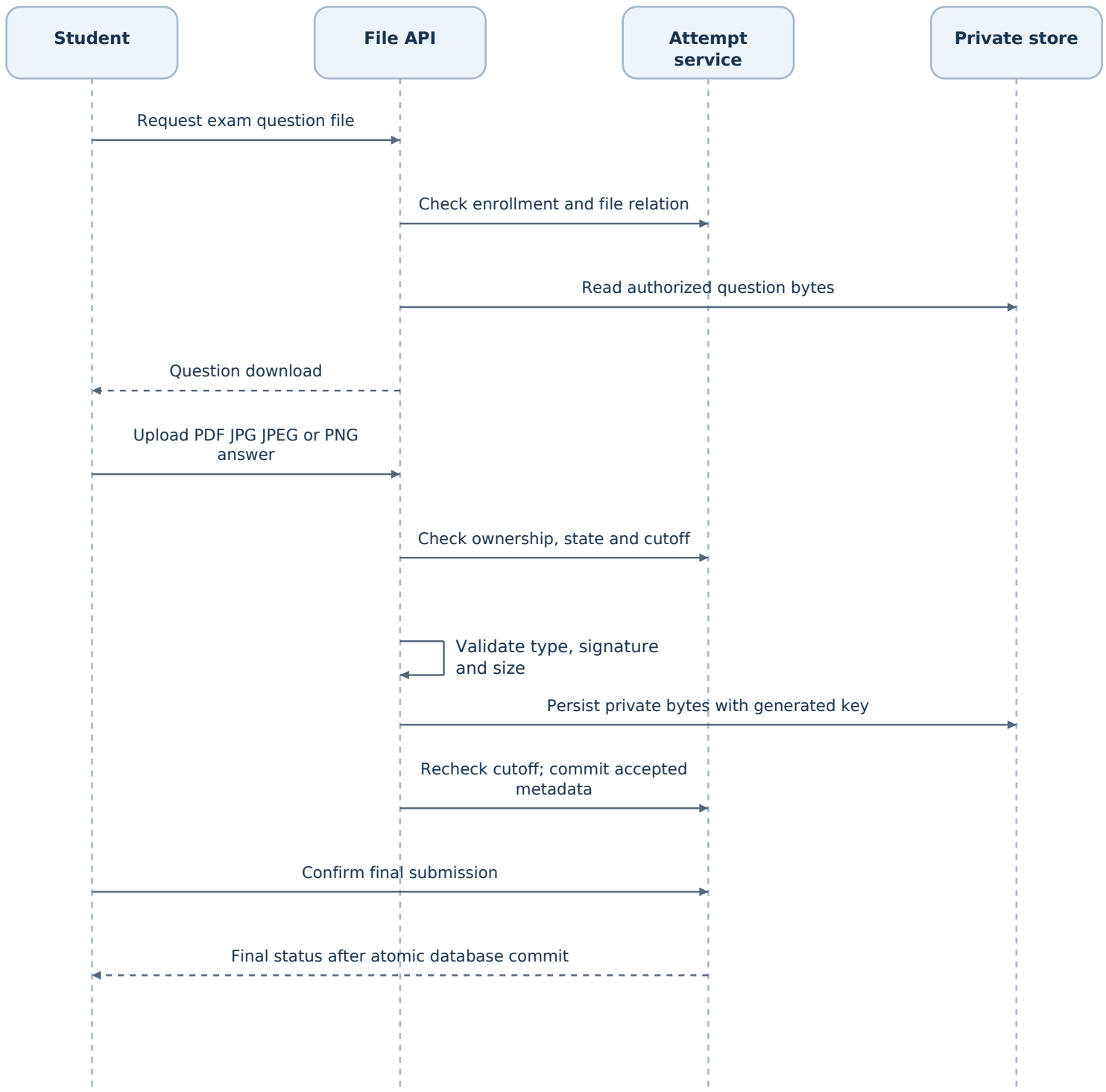
UML sequence view | Solid arrows are requests; dashed arrows are responses. Ordered from top to bottom.



Any failed ownership, membership, state or cutoff check rejects the write. Deadline processing uses the same finalization service for MCQ. A repeated submit returns the existing final state and never creates a second grade.

Figure 9 Descriptive submission sequence

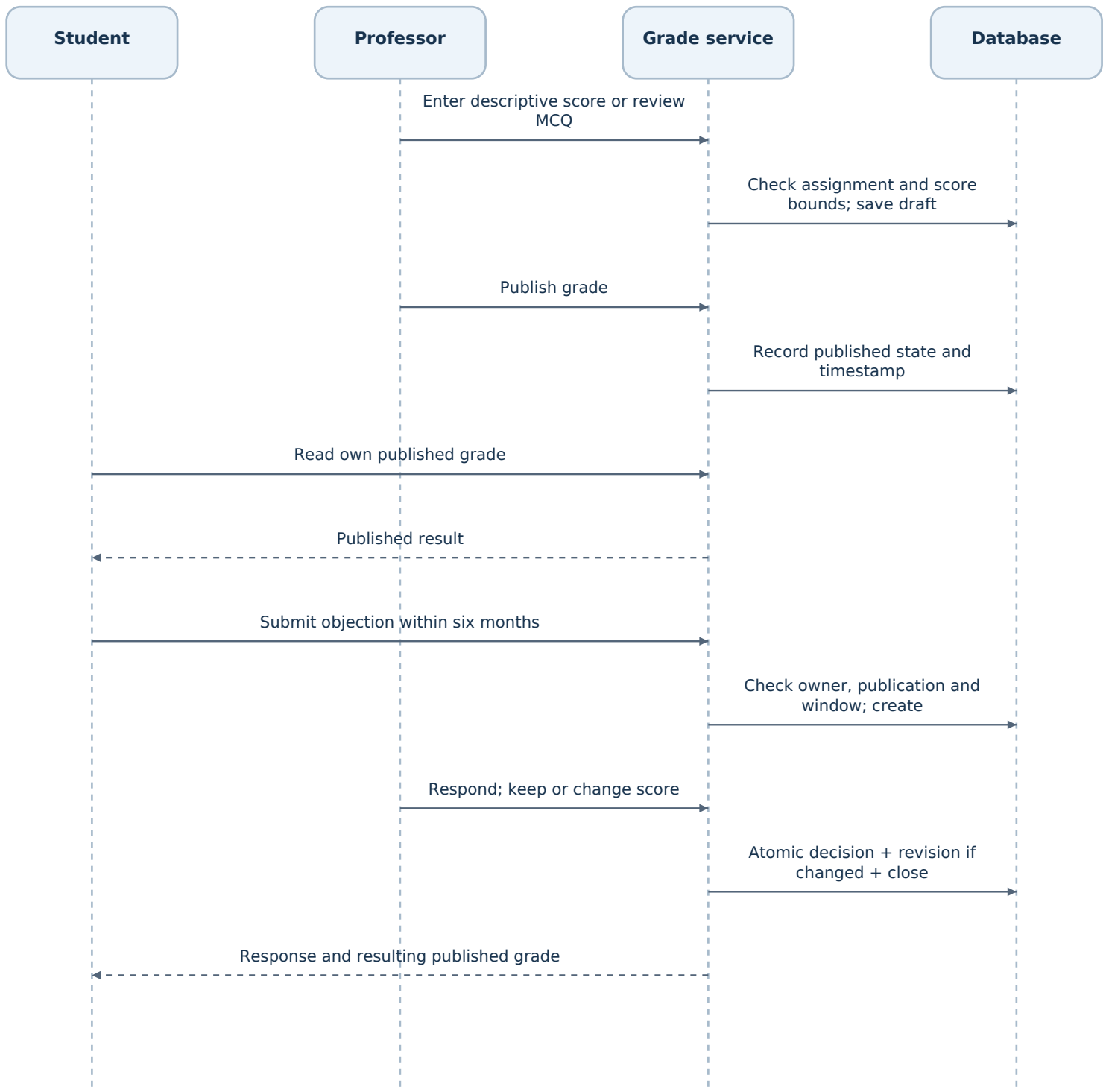
UML sequence view | Solid arrows are requests; dashed arrows are responses. Ordered from top to bottom.



Attempt service uses PostgreSQL for metadata and final state (omitted as a fifth participant for readability). If final acceptance fails, clean up unlinked bytes safely. No accepted answer at cutoff means absent; accepted but unsubmitted files require an explicit policy.

Figure 10 Grade publication and objection

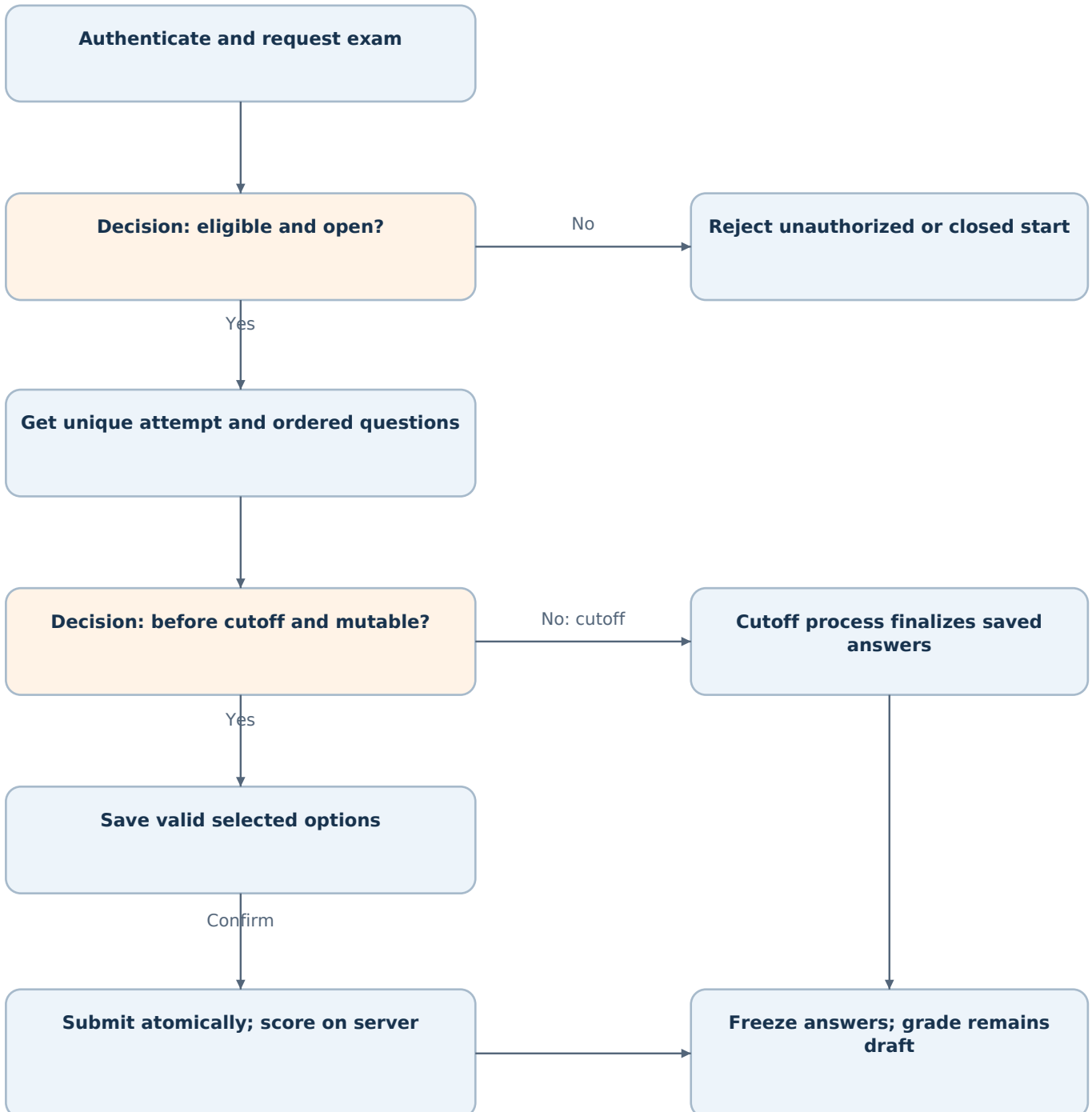
UML sequence view | Solid arrows are requests; dashed arrows are responses. Ordered from top to bottom.



Changing a score requires a revision with actor, reason, old and new value. Keeping a score requires a response but no fabricated score change. Objection eligibility uses the publication timestamp and the confirmed calendar policy.

Figure 11 MCQ activity

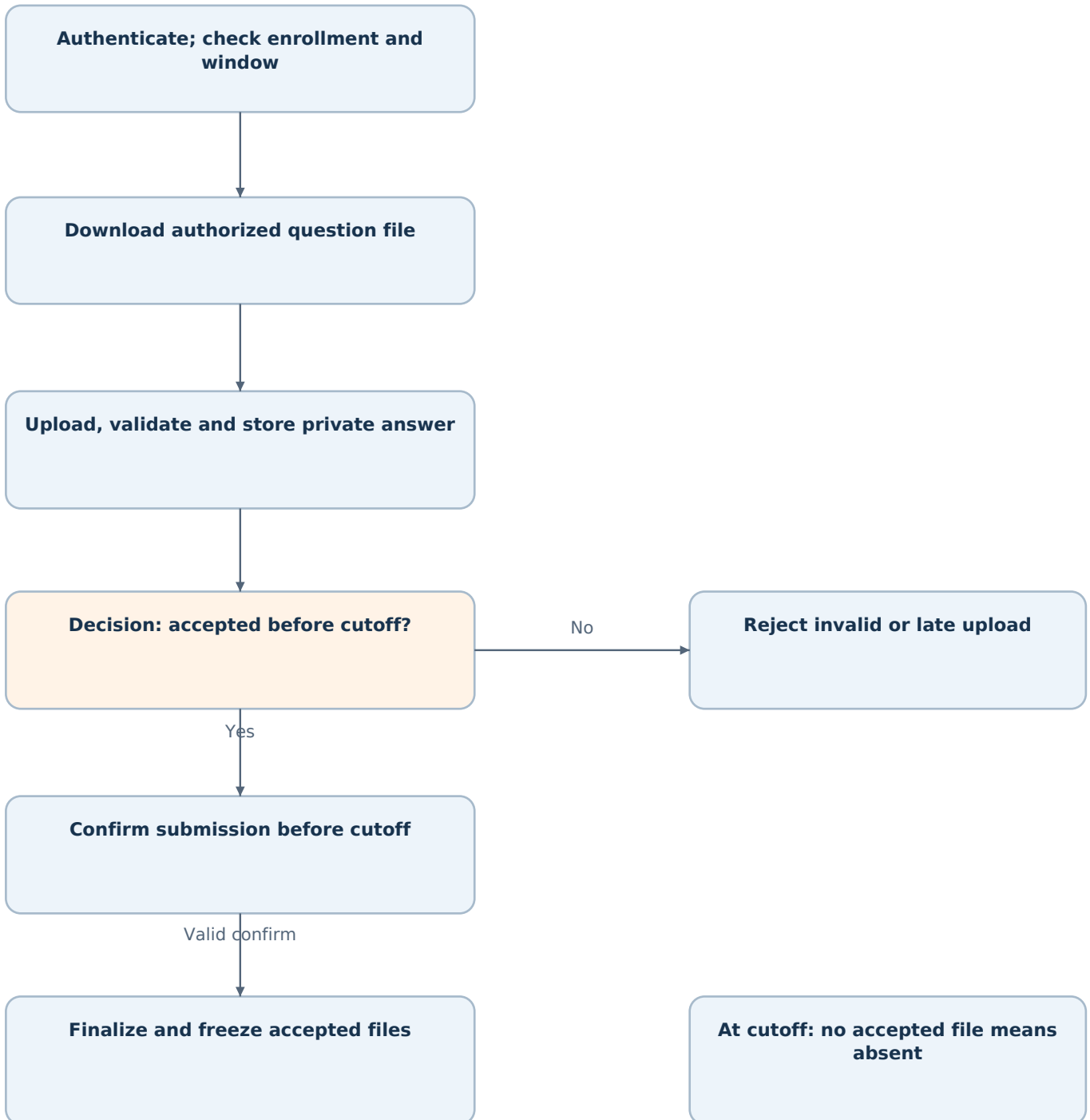
Activity view | Branch labels express server evaluated conditions.



Saving may repeat while the attempt is mutable; navigating does not finalize an attempt. If already finalized, return the existing status. Both finalization paths serialize against saves, score only committed answers and leave missing questions unanswered.

Figure 12 Descriptive activity

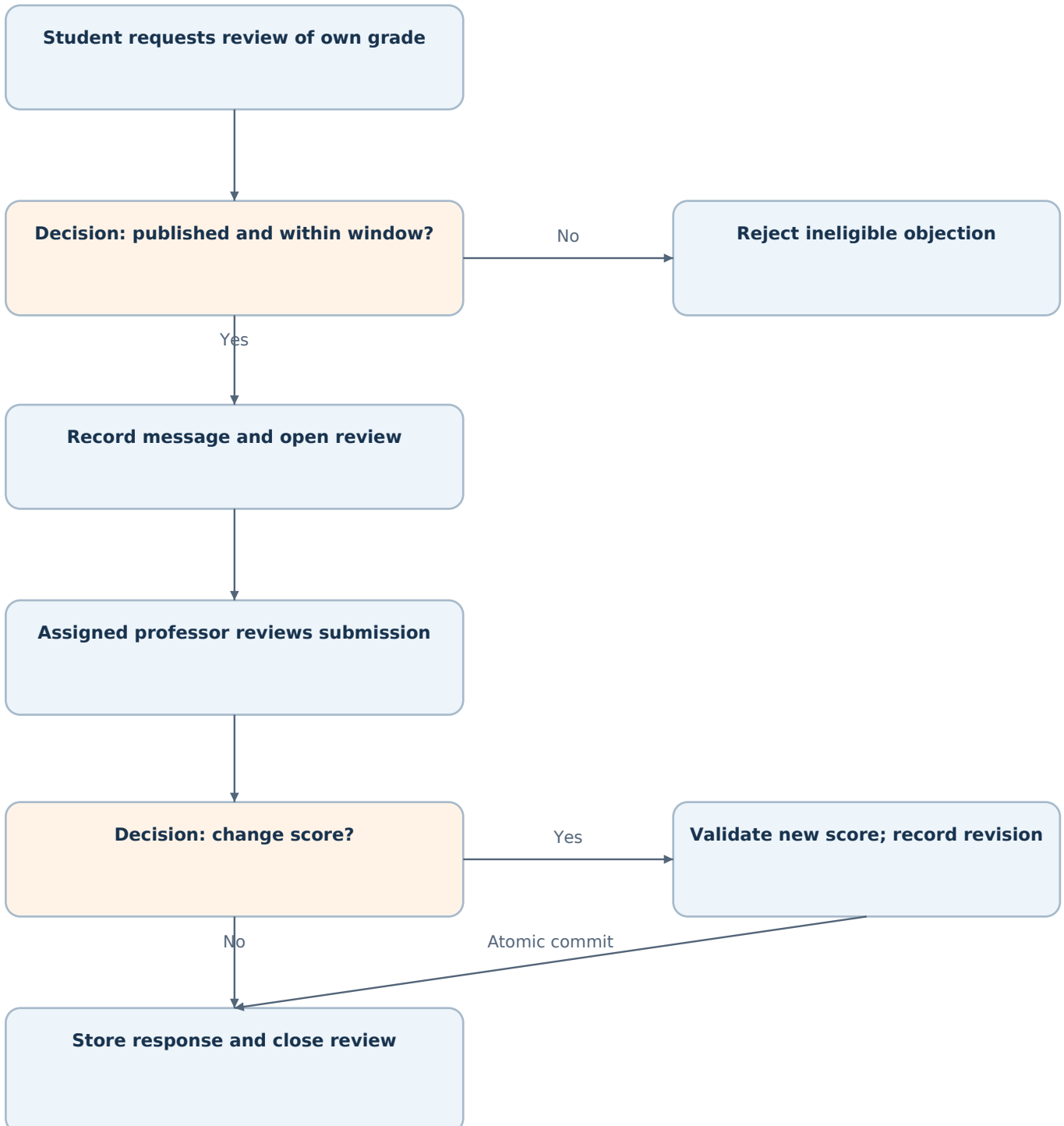
Activity view | Branch labels express server evaluated conditions.



Acceptance is recorded only after successful validation, private persistence and a final deadline check. Uploaded but unsubmitted accepted files have a separate unresolved cutoff policy; the absence condition is specifically no accepted file.

Figure 13 Objection activity

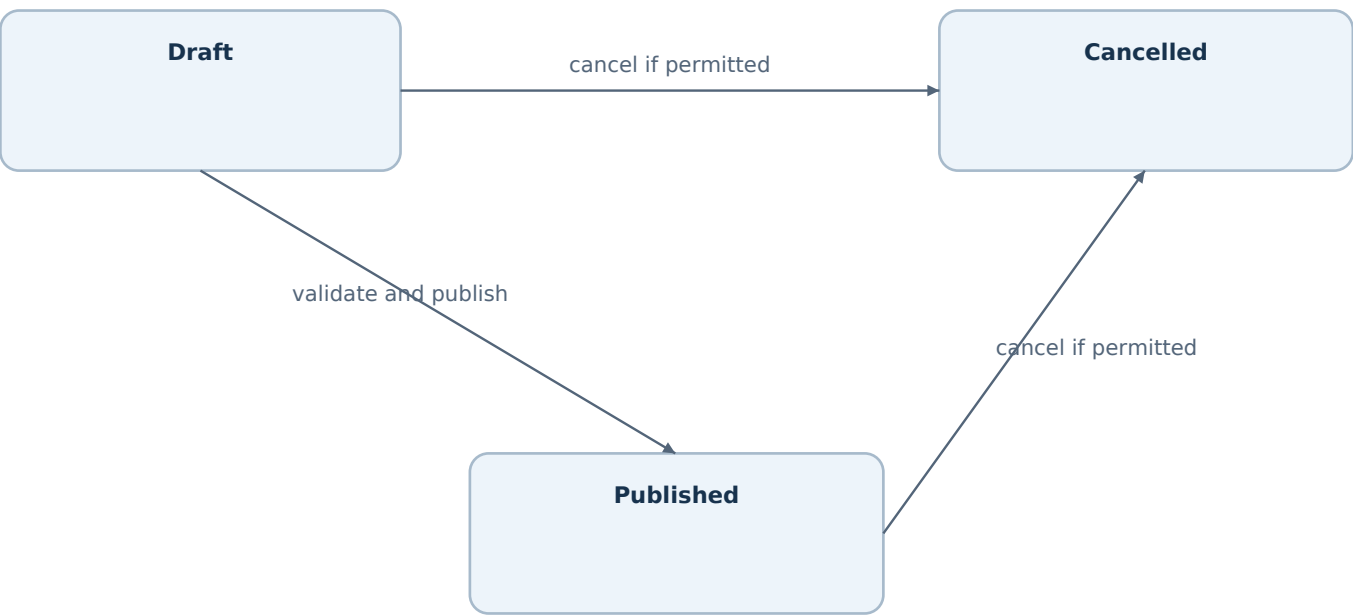
Activity view | Branch labels express server evaluated conditions.



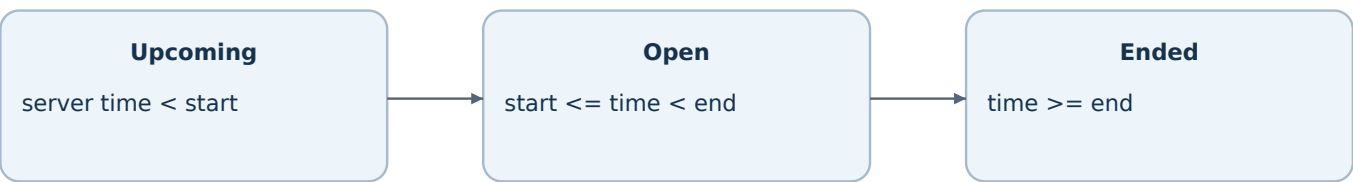
Authorization applies to opening and resolving the review. A revised score, append only grade revision, decision and objection status commit in one transaction. The student sees the recorded response and resulting published grade.

Figure 14 Exam lifecycle

State view | The stored design status and availability derived from time are different concepts.



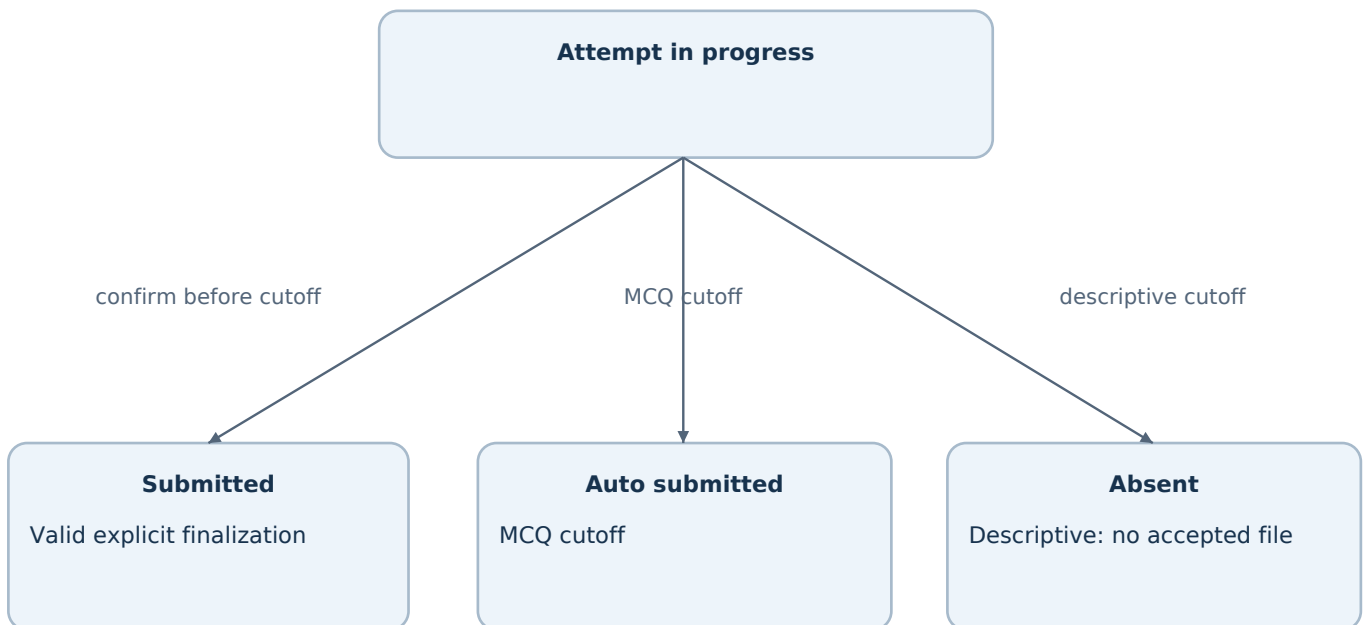
Availability while published



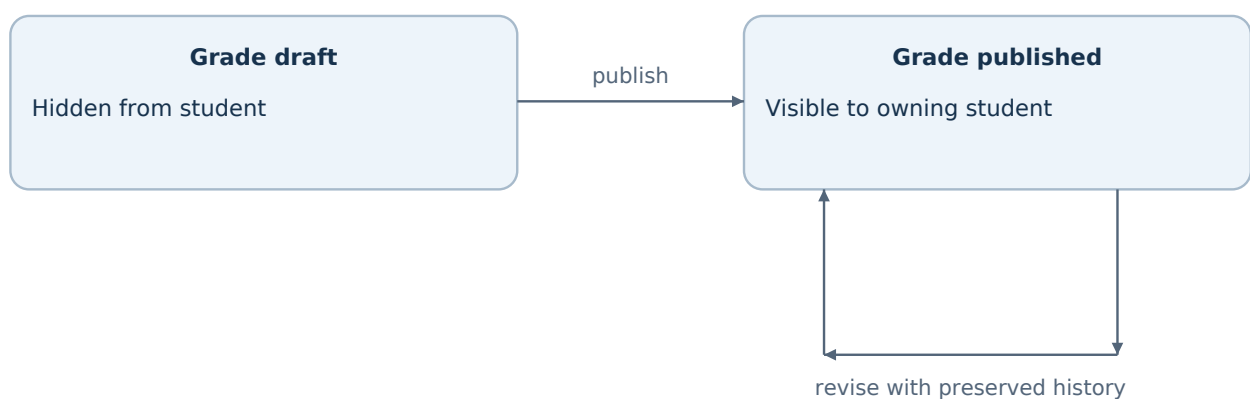
Upcoming, open and ended are derived availability phases, not asserted database enums. An individual attempt may end earlier because of duration. A manual close operation must be mapped to the actual schema and must preserve existing academic records.

Figure 15 Attempt and grade lifecycles

State view | Business state names are symbolic and must be mapped to the current Prisma enums.



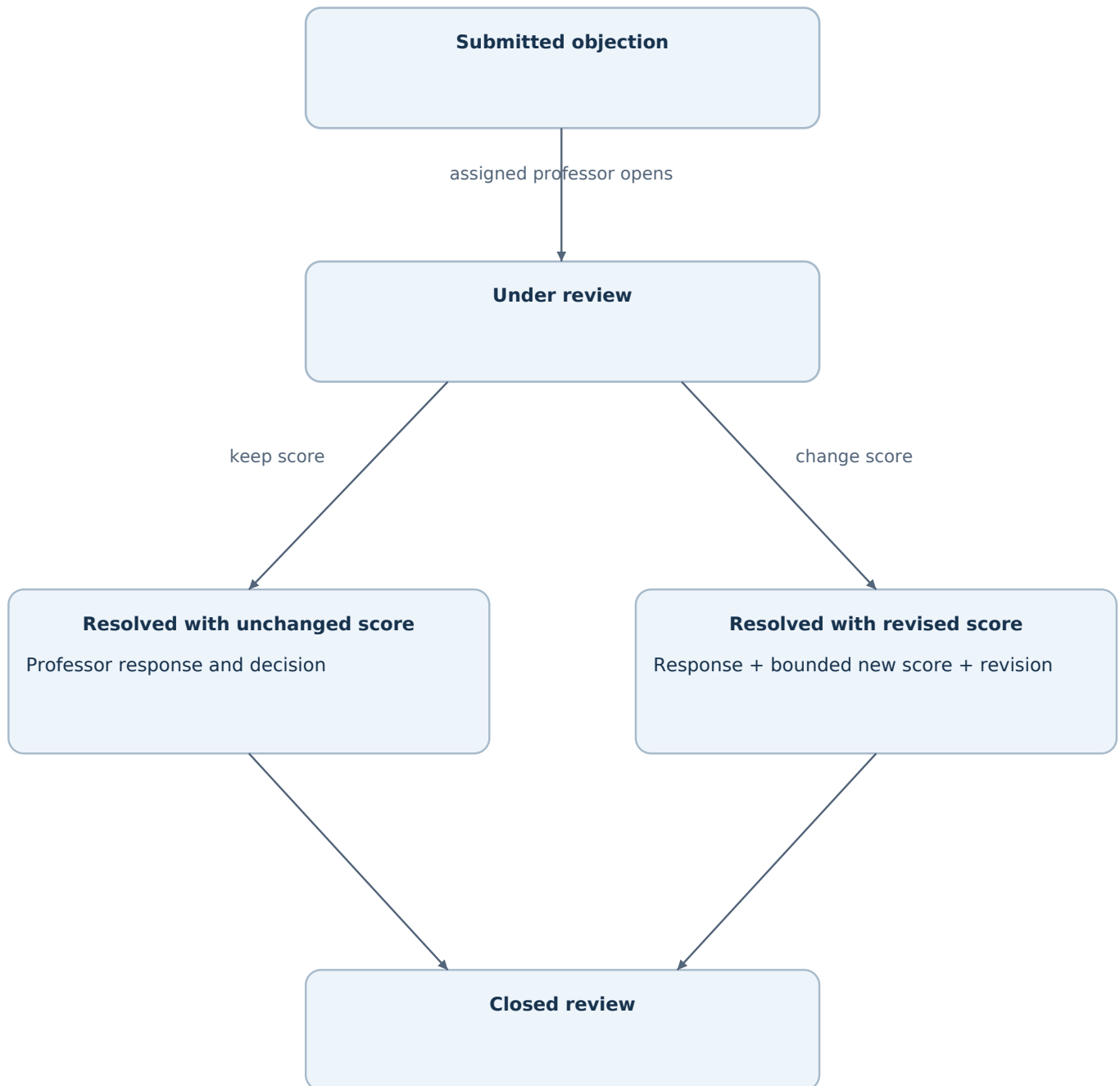
All final attempt states reject answer changes. The accepted but unsubmitted descriptive case needs a separate confirmed transition.



MCQ finalization produces a draft grade. Descriptive grading requires professor review. Publication does not imply unlimited revision authority; every change checks professor scope and records the prior score.

Figure 16 Objection lifecycle

State view | Response and revision history are preserved when the review is resolved.



Resolved and closed may be represented by one enum in the implementation. Reopening or multiple review rounds are not approved by this figure. The six month rule applies to creating an objection; it does not invent a deadline for the professor's response.

13 Testing strategy

Use unit tests for deadline computation, grading, score bounds, objection eligibility and transitions. Use integration tests against an isolated PostgreSQL instance for constraints, transactions, finalization races and file metadata. API tests validate authorization and response projections. Frontend tests cover Persian RTL behavior, forms, navigation, countdown and upload feedback.

Case	Expected result
T01 Cross student access	Another student's attempt, grade and file remain inaccessible.
T02 Cross professor access	An unassigned professor cannot edit exams, publish grades or resolve objections.
T03 Deadline boundary	Writes immediately before cutoff are accepted if committed as valid; at and after cutoff are rejected. Client clock changes have no effect.
T04 Submission race	Concurrent save and submit produce one final outcome; no postfinalization edits or duplicate grades.
T05 Answer secrecy	Pre submission questions omit correct flags and answer keys.
T06 File safety	Invalid type, mismatched signature, excessive size and traversal filenames fail without accepted metadata.
T07 Grade visibility	Draft grade is hidden; own published grade is visible.
T08 Objection boundary	Wrong owner, unpublished grade and expired window fail. Changed score preserves history.

13 End to end validation and evidence

Scenario	Path and assertions
MCQ	Student login → eligible exam → start → navigate and save → submit → backend score → professor publication → student grade → objection → professor response. Verify each actor sees only permitted records.
Descriptive	Question download → validated private upload → confirm → professor file review → enter score → publish → student result. Verify cutoff behavior and authorized download.
Recovery	Retry cutoff processing, refresh rotation, repeated final submit and failed file acceptance. Verify no duplicate grade and no exposed private bytes.
Persian UI	RTL form layout, question navigator, readable errors, Solar Hijri dates and mobile screens.

Execution record For each test, record commit, environment, input, expected result, actual result, pass or fail and supporting evidence. This document is a test strategy, not a claim that tests were executed.

Priority for submission Provide actual API paths, current schema constraints, a successful reproducible startup guide and representative screenshots. Disclose known frontend or runtime failures instead of claiming completion.

14 Deployment requirements

- Record supported runtime and package versions from project manifests. Configure PostgreSQL, JWT secrets and allowed frontend origins through environment variables.
- Use reviewed Prisma migration history. Preserve the existing database; do not apply a conflicting bundled initial migration or reset it. Generate Prisma Client after schema alignment.
- Provision demonstration accounts and academic assignments through the project's controlled seed or administrative setup. Do not include real passwords in public documentation.
- Configure private storage permissions, upload bounds and authenticated download routes. Document the actual storage path or provider privately where appropriate.
- Use absolute timestamps and explicit Asia/Tehran conversion. Configure a reliable cutoff process and safe retry behavior; do not depend on the browser countdown.
- Document backend and frontend ports to avoid collision. Provide HTTPS for deployed access, backups, restoration procedure and secret redaction.

The delivery package should include README, .env.example, schema and migration files, API reference or Postman collection, actual test results and a source repository link. Exclude .env secrets, node_modules and real student data.

15 Future improvements

- University single sign on and centrally managed account lifecycle.
- Approved accommodation rules and controlled multiple attempt policies.
- Notification delivery for published grades and review decisions.
- Stronger malware scanning and documented file retention policy.
- Automated deployment, backup restoration checks and accessibility review.

These items are proposed extensions. They should not be presented as implemented features without source and execution evidence.

Remaining reconciliation Confirm the six month calendar interpretation; map lifecycle labels and entity fields to Prisma; settle uploaded but unsubmitted descriptive answers; document actual deadline scheduling and exact API contracts.

16 Conclusion and revision notes

The design connects university provisioned identity and course eligibility to timed participation, secure answer handling, deliberate grade publication and accountable objection review. Its essential controls are resource scoped authorization, server enforced cutoffs, private files and atomic academic state changes.

Revision	Improvement
Database figures	Split academic setup, participation and grading history into readable models with explicit cardinalities.
Sequence figures	Separate API, service and persistence responsibilities; include answer secrecy, finalization and private file acceptance.
Activity figures	Expose eligibility, cutoff and grade revision branches with the unresolved descriptive policy identified.
State figures	Replace transition tables with lifecycle views; distinguish time derived exam availability from stored status.
Document layout	Consistent A4 pages, readable captions, vector diagrams, page numbering, navigation bookmarks and a real contents guide.